Study of Sigstore integration into open-source repository security and end-user side security in the modern software supply chain

Noah deMers
Core4ce
University of Hawaii at Manoa
demersn@hawaii.edu

***Abstract*— Current security standards in the software supply chain prove to be inadequate, particularly involving the use of open-source, third party repositories (Okafor et al. 2024). Many of these repositories do not have sufficient rigor for intaking valid credentials from developers who wish to upload artifacts to the repositories, and even if they do, sufficient verification of those credentials is lacking (Liu et al. 2020). Latest literature reviewed in this work also highlights the lack of verification conducted on the end-user side, where an end user who wishes to use open-sourced software retrieves it from the open-source repository (Okafor et al., 2024; Schorlemmer et al., 2024). Numerous problems have been highlighted in the current security measures in the open-source repository data transfer, including key management and the lack of adequate verification (Okafor et al. 2024, Newman et al. 2022). This work proposes the integration of Sigstore, an open-source toolset that signs and verifies code, and additionally logs the certificates into a public, append only log. Sigstore uses ephemeral keys, which mitigates many of the problems currently faced with the security of open-source repository use. This work will examine the hesitancy of Sigstore adoption across tech sectors and then propose how it can be used to strengthen security measures in the modern software supply chain involving open-source repository use.**

This work also conducts an experiment that compares the security of a traditional public-private key pair to a Sigstore integrated security measure. This is measured by first simulating a stolen key in both environments, then comparing how many malicious artifacts are accepted in the simulated repository- for security reasons a real open-source repository will not be used- how many malicious artifacts are accepted by the end user, and how long both environments take to detect malicious use.

1 Introduction

The use of open-source software is becoming increasingly more common in numerous sectors, both public and private. (Schorlemmer et al. 2024). This is evident in the 2025 report by the Synopsys that found that 97% of codebases across 16 industries incorporated open-source components (Synopsys, 2025). Many developers rely on open-source repositories to efficiently build necessary software, rather than building everything from scratch. This has greatly increased not just productivity but also the scalability and capabilities of projects/products. However, the use of open-source repositories also increases attack surfaces; because of the wider software supply chain, attackers now have more avenues to push malicious code into the software (Sonatype, 2024). Other literature has focused on numerous ones such as pushing container images with vulnerabilities (Liu et al., 2020), typosquatting and name confusion attacks (Sonatype, 2024) among others. In sectors like the DoW, the middle of the software supply chain is already considered a secure environment (P. Ritchey, personal communication, October 1, 2025). Thus, this work aims to reduce the security risks at both at the beginning repository level—where malicious packages may be accepted—and at the end-user level, where verification is inconsistent. Open-source software distribution relies heavily on open repositories (eg PyPI, npm) to host libraries used by countless sectors Synopsys (2025). Attackers with compromised credentials can upload malicious code that appears legitimate, and end users often have limited means to verify authenticity. While ephemeral signing and transparency logs (eg Sigstore) improve code provenance, current measures prove insufficient. Current measures create risks of security breaches despite cryptographic signing (Schorlemmer et al. (2024). This work will elaborate on the design, implementation, evaluation of integrated open-source measures.

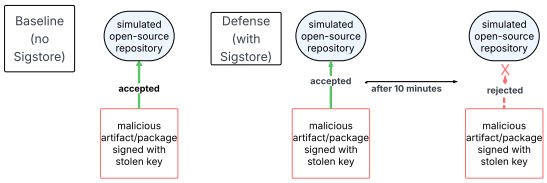


Fig. 1. pipeline of the repository side experiment.

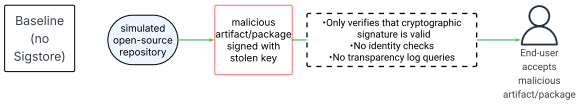


Fig. 2. Pipeline of the baseline for the end user side experiment.

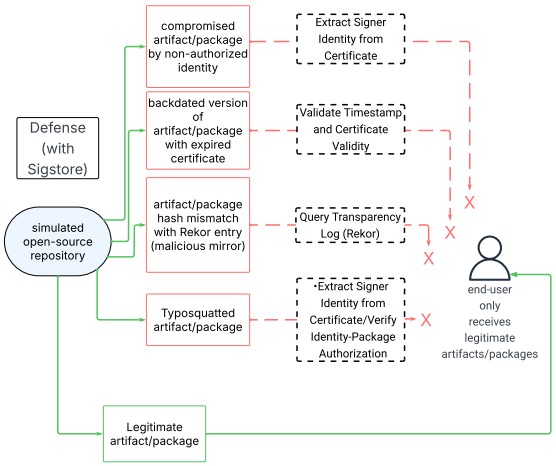


Fig. 3. Pipeline of the Sigstore-incorporated defense for the end user side experiment.

1.1 Measures

The measures for the experiment are: securing the repository side by enforcing a Key Authorization Map (KAM) that limits which publishers can upload packages, using ephemeral keys and transparency logs (Sigstore: Fulcio + Cosign + Rekor) to provide end-to-end artifact provenance, and finally, implementing end-user verification to ensure only authorized, logged artifacts are accepted.

1.2 Research Question

What are the measured effects of adding registry-side enforcement (KAM + mandatory sigstore signing + Rekor checks + optional multi-sig/attestations) together with end-user enforcement of ephemeral-key signatures (Sigstore) to reduce the window of undetected malicious package acceptance and increase the detection of malicious actions versus current private-public key pair measures?

1.3 Significance of the Study

In web infrastructure the standard for data transfer being conducted under encryption is already common. Most average users would be hesitant to interact with an HTTP site over an HTTPS site (Kerschbaumer et al. 2025). However most open source third party repositories do not have nearly as robust security measures when they receive uploads (Ruohonen et al 2021; Schorlemmer et al., 2024). This work aims to set this same standard used in web infrastructure in the use of open-source libraries, particularly at the very beginning, where developers upload their artifacts to the repositories, and at the very end, where end users can verify legitimate ownership of that artifact. With the current standard for data transfer between open-source repositories, it is essentially akin to performing data transfer within the web infrastructure in plain text, which poses obvious security risks.

1.4 Key Terms

Several key terms used extensively throughout this work have been defined below. Much of these terms are directly related to key cybersecurity concepts relevant to this work, and a thorough understanding of these terms will deepen the comprehension of this work moving forward.

- **Certificate Authority (CA):** A certificate authority (CA) is a third-party organization trusted to validate the identity of entities—ranging from websites, businesses, and even individuals—and to issue electronic documents also known as digital certificates. (National Institute of Standards and Technology, 2020)
- **Continuous Integration/Continuous Development (CI/CD):** a software development practice that uses automation to continuously integrate, test, and deliver or deploy code changes (National Security Agency & Cybersecurity and Infrastructure Security Agency, 2023)
- **Ephemeral:** lasting a very short time. In the context of cybersecurity, ephemeral keys are short lived cryptographic keys. (Newman et al., 2022)
- **Hardware Security Module (HSM):** physical, tamper-resistant device used to store and manage cryptographic keys. (National Institute of Standards and Technology, 2020)
- **Hypertext Transfer Protocol (http):** the primary protocol used to send data between a web browser and a website. Unlike https it is plaintext and not encrypted (Kerschbaumer et al., 2025)
- **Hypertext Transfer Protocol Secure (https):** the encrypted primary protocol used to send data between a web browser and a website (Kerschbaumer et al., 2025)
- **Open repository:** a public, central location for developers to upload their code/projects for other developers (the end-users) to use.
- **OIDC (OpenID Connect):** an identity layer on top of OAuth 2.0 that enables users to authenticate with identity providers and verify their identity without requiring long-term private key management. (Kalu et al., 2025)
- **Public Key Infrastructure (PKI):** The overall system used to manage the digital identification/security verification of data transfer in the web space. (National Institute of Standards and Technology, 2020)

2 Review of the Literature

In this section a review of the current literature was conducted, including the background of current web PKI security practices as well as Sigstore, the literature search strategies, the problems faced in current security practices, and the lack of widespread Sigstore adoption.

2.1 Background

The current method of ensuring software provenance, particularly in web PKI is to have the developer of the code create a public-private key pair. The developer then hashes their code, using their private key to sign the hashed value of the code. (Rekow et al 2021). Then the developer sends their public key and proper identification to a CA. The CA verifies the developer's identity, then issues a digital certificate; this verifies that the public key is indeed tied to that developer. The digital certificate includes the developer's public key, developer identification, and other metadata about the CA itself , such as the CA's name, CA's public key, signature algorithm used, etc (Newman et al., 2022). The CA then hashes this certificate, then signs the hashed value of the certificate with their own private key. This way when an end user accesses a site, they first check to see if the certificate is legitimate and has not been tampered with by using the CA's public key. (Rekow et al., 2021). After successfully verifying the legitimacy of the certificate, the end user then uses the developer's public key in the now vetted certificate to start an encrypted connection. However, there is a problem with this key infrastructure. With this method developers and CAs alike would have to manage key pairs which could be compromised (Kalu et al., 2025). This is further elaborated in the problem: key management section later in this paper. Now this work will turn to the current key infrastructure of open-source repository data transfer.

Even in high security operations, open-source use is highly unavoidable. With the ever-increasing speed, scale, and complexity of software advancement, it is unfeasible for developers to code the entirety of the software without the use of open-source libraries. Current literature shows that despite the prominent use of open-source repositories, many organizations do not implement sufficient security measures (Synopsys 2025). Another topic that will be discussed will analyze the current industry-both public and private- standard.

Sigstore, an open-source set of tools, uses ephemeral keys to sign and verify an artifact. First, the developer would run Cosign, the ephemeral key pair generation tool for Sigstore, then sign into their OIDC such as GitHub account, Google account etc (Newman et al, 2022). The developer would then get an ephemeral private-public key pair that lasts only ten minutes. Then, Fulcio, Sigstore's code signing CA, issues an ephemeral certificate, that contains the developer's identity and ephemeral key; similar to any other X.509 certificate. (Newman et al., 2022). Cosign then automatically executes the hashing of the artifact, followed by the signing of the hash with the ephemeral private key. Sigstore then uses Rekor, its signature transparency log. Sigstore sends the essential data pertaining the signed hash value of the artifact, the short lived Fulcio certificate, and other meta data including the artifact digest, the signature, log entry ID, log index, inclusion proof, timestamp, certificate chain, OIDC issuer URL, subject identity, CI/CD workflow information, and root hash and tree size for log verification. With this Rekor is able to log everything. Now even after the developer's ephemeral key pair expires, the Rekor log of their artifact will include the public ephemeral key at the time, allowing the end user to verify (Newmann et al., 2022). This eliminates the security risk of an attacker gaining access to the developer's private key and uploading malicious code posing as the developer. It should be noted that a standard private key could be harder to compromise compared to an OIDC login that would be used for Sigstore; namely that it could be stored directly on external hardware (National Institute of Standards and Technology, 2020). However, the OIDC is a simple login and password that could be compromised, especially if a developer is using an easily crackable password. Despite that, this study is still conducted around the use of Sigstore. It is deduced that the trade-off between Sigstore's more vulnerable access to the key pair, is that since it is ephemeral the damage surface is minimal compared to the event that an attacker gains access to a standard private key pair (Kalu et al., 2025I). In other words, an attacker only has a short time to perform any malicious acts before they lose access. Not only that but with Rekor's transparency log built into the whole key infrastructure, malicious use can be detected and resolved. The Problem: Verification section delves into the problem of the lack of verification in standard practice across sectors (Schorlemmer et al., 2024). The implementation of Sigstore would addresses that problem.

Most of the current open-source repositories do not have adequate safety measures against attackers uploading malicious code to the repository (Schorlemmer et al 2024). This work proposes a way for repositories such as PyPI and the like to both increase security in their repository –i.e. preventing malicious code from being uploaded- and the detection of that malicious code, elaborated in the Open-Source Repository Side Security section to follow. This work also contributes to security measures on the end-user side for additional security in the event that the aforementioned security measures proposed for the repositories proves inadequate.

2.2 Literature Search Strategies

In order to search for relevant, properly peer reviewed sources, this work utilized Perplexity AI to gather sources. That was much more efficient than manually searching through scholarly articles at sources such as Google Scholar etc, as many of them are not properly peer reviewed yet. In addition, Perplexity AI is much better at finding relevant sources in a timely manner. This literature search strategy has introduced key literature that emphasizes the problems highlighted below.

2.3 Problem: Key Management

An interview-based study conducted on the sector-wide issues on web-An interview-based study conducted on the sector-wide issues on web-based PKI found that key management were the most reported issues; this is anywhere from key generation, key management, and key distribution (Kalu et al 2025). As the PKI process is the same for

open-source repository signing, this problem is persistent in modern software supply chains as well. A traditional key pair in open-source repository data transfer proves to be a bottleneck as there are mitigations necessary in the case of lost/compromised keys (Merril et al. 2023). In particular, if an attacker gains access to a developer's key pair, the attacker can masquerade as the developer to push malicious code. A study conducted in South Korea found that 455 certificates were compromised by attackers to push malware; those certificates were meant for legitimate South Korean companies (Kwon et al. 2021). Note that the CA private key is much less likely to be compromised; the National Institute of Standards and Technology (NIST) regulates the private key management to be done by more than one individual and key generation to be done under higher security scrutiny, including the use of HSM models (NIST, 2020). While developers can also in theory implement these security measures, in practice across sectors it is unrealistic to do so.

2.4 Problem: Verification-or lack thereof

Even when developers in organizations sign software, it is rarely used to verify provenance, despite many of them believing in their uses; in the case of external contributors, not a single signature was verified (Kalu et al., 2025). Without proper security verifications in place, all an attacker would have to do is compromise a key pair of a developer to push malicious code. With current standards, the end user simply trusts the public key of the developer whose artifact they are using that they obtained from the repository; this blind trust poses security risks as the repositories themselves may not be secure (Merrill et al., 2023). It was aforementioned that the CAs for websites are under stricter security regulations by the NIST than open-source repositories are, who are not under the same regulations. So, in addition to the lack of verification of the developers posing a security risk, as it stands, current open-source repositories themselves are not a trustworthy source, precisely because they do not properly verify developers uploading to them (Schorlemmer et al. 2024).Verification of an incoming signature- at the beginning, repository side of the software supply chain- and at the end user step in the software supply chain would greatly improve software provenance. Other literature has demonstrated the lack of verification of open-sourced software that almost every developer in tech sectors use in their work (Kalu et al., 2025). Sigstore's built in verification would not only directly mitigate this security risk, but it is also easy to implement, proving low overhead (Newman et al., 2022; Kalu et al., 2025).

2.5 Lack of Sigstore Adoption

Despite much of the cumbersome steps in the encryption process being handled automatically by Sigstore, and the cost-free access to it, it is still not widely adopted by tech sectors to make encryption data transfer the standard, similar to what HTTPS is to web security. In a use study conducted on Sigstore, several issues regarding Sigstore were concluded (Kalu et al 2025). Some issues, such as integration to other systems, is addressed in this study. Other issues involving the lack of developer privacy to the transparency log (Kalu et al 2025), are addressed in the Speranza paper (Merril et al 2023). A major factor in the hesitance to Sigstore adoption is the delayed latency time (Kalu et al 2025). To mitigate this, we use private instances of the Rekor log instead of the global public Rekor log, which increases latency. However, in a real-world scenario, the Rekor log would have to be public for everybody to see. The use of a public log could have latency issues. Future work could address the latency issues. Sigstore has several additional external factors impeding its widespread adoption, such as a lack of an active user community, thorough documentations, and regular updates (Kalu et al 2025).

2.6 Open-Source Repo Side Security

With that, this work also reviews other literature on the beginning, repository side of the software supply chain, since this study aims to implement Sigstore in the repository side as well as the end user side later mentioned. Current DockerHub measures do not meet adequate verification standards for the uploaded images (Liu et al 2020). One study found that 46% of analyzed PyPI packages had a security issue (Ruohonen et al, 2021). Out of all the malicious open-source packages in 2024, 98% of them were npm packages (Sonatype, 2024). Several of these widely used repositories do not require signature/verification for uploading. This would be equivalent to sending website data or the internet in plain text. As aforementioned, users' standard for web security is that data transfer is encrypted.

2.7 End-User Side Security

As elaborated under the section "Open-Source Repo Side Security", end users- developers in sectors that are about to use an open-source artifact- should not be so reliant on the repository to implement sufficient security measures. This work provides a method for end-developers and users to do a final security check on the open-sourced repository to ensure security using Sigstore. Other literature shows most developers also do not practice adequate end user- security measures. A study conducted found vulnerabilities in 60 projects, and out of those, only 22 developers responded (Kabir et al 2022). End user developers tend to disregard potential security breaches as false positives (De Sousa et al 2025). In addition, they frequently do not adequately check the open-source artifact they are installing (Kabir et al., 2022). This brings increased risk as these end users are not assuming a zero-trust role when obtaining open-sourced artifacts-ie,, they are blindly trusting that the artifact from whatever presumably trusted repository has not been compromised (Kalu et al. 2024). As mentioned, although this work contributes to increased security of data transfer on the open-source repository side, it must be noted that equally through security measures on the end-user side is essential for the purpose of normalizing the open-source data transfer encryption standard. The aforementioned repository side security measures are not foolproof, and introducing

security measures on the end-user side would reduce the attack surface in the modern software supply chain.

3 Method

The purpose of this work is to evaluate the effectiveness of integrating Sigstore's ephemeral key infrastructure with Key Authorization Map (KAM) enforcement at both the repository and end-user levels of the open-source software supply chain. This study addresses the problem of inadequate security measures in open-source repositories by implementing and testing a defense-in-depth approach that combines repository-side security measures through ephemeral keys with client-side verification through transparency logs and identity-based authorization policies.

The conceptual framework builds on existing public key infrastructure research while addressing its limitations. Traditional PKI relies on long-lived key pairs that create operational overhead and security vulnerabilities when compromised (Merrill et al., 2023). Sigstore's ephemeral key approach provides a time-limited window for potential attacks while maintaining cryptographic verification (Newman et al., 2022). This study implements Key Authorization Maps at the repository level and identity-based authorization policies at the client level to create multiple defensive layers against supply chain attacks.

3.1 Research Method and Design

The research method is quantitative experimental design, which is appropriate for controlled comparison of security measures through measurable outcomes such as detection rates, false positive rates, and verification latencies as a quantitative experimental design is a systematic approach to research that tests for causal relationships by manipulating an independent variable to see its effect on a dependent variable, while controlling other factors. The study employs two primary experiments comparing baseline configurations (traditional long-lived keys) against defense configurations (Sigstore ephemeral keys). The first experiment tests repository-side security where packages are uploaded, while the second tests end-user verification where packages are downloaded. This two-layer approach reflects real-world software supply chain architecture and demonstrates defense-in-depth effectiveness.

3.2 Population and Sample

The target population consists of software package transactions in open-source repositories under potential compromise scenarios where attackers have obtained legitimate publisher credentials. For the repository-side experiment, 12 trials were conducted: 9 in defense mode with varying time-to-live (TTL) values (600 seconds) and theft delays (0, 300, and 700 seconds), and 3 in baseline mode. The sample demonstrates how ephemeral key expiration limits the temporal attack window. For the end-user experiment, 90 verification trials were conducted: 45 in each configuration (baseline and defense). The sample included 40 malicious package scenarios distributed across four attack types (10 trials each): compromised packages, backdated packages (rollback attacks), malicious mirror substitutions, and typosquatting attempts. Five legitimate package trials served as a control group. This systematic sampling covers diverse attack vectors documented in current literature (Sonatype, 2024; Liu et al., 2020).

It is important to know that although the attack vector of stolen keys was emphasized throughout this work, this is not tested in the end-user experiment; this is because stolen key attacks can only be prevented at the repository side through ephemeral key expiration, while end-user side verification cannot distinguish between legitimate publishers and attackers using stolen authorized keys.

3.3 Data Collection Instrumentation and Procedures

In this subsection the methods of data collection for both the repository-side experiment and the end-user side experiment are discussed.

3.3.1 Repository-Side Experiment:

The data collection instrument consisted of a simulated package registry implemented in Python 3.11 incorporating: an upload verification service validating signatures and checking ephemeral certificate expiration, a Key Authorization Map service enforcing publisher-package authorization policies, and a Rekor transparency log recording all signing events. The data collection procedures were: (1) trial initialization with attacker-generated malicious packages and compromised credentials, (2) temporal simulation with defined theft delays testing time-based security guarantees, (3) upload verification checking signature validity and certificate expiration status, and (4) automated data recording of verification outcomes, detection mechanisms, and artifacts accepted before key expiration.

3.3.2 End-User Side Experiment:

The data collection instrument consisted of a package verification system with two modes: baseline (cryptographic signature verification only) and defense (full Sigstore verification including transparency log queries, identity validation, temporal checks, and policy authorization). Data collection procedures were: (1) attack scenario generation creating packages for four attack types plus legitimate control packages, (2) policy engine configuration mapping certificate identities to authorized package names, (3) verification execution performing baseline or defense checks sequentially on each package, and (4) automated data recording of verification outcomes, security check results, and verification latencies. Both experiments were fully automated, eliminating observer bias and enabling precise timing measurements with all timestamps recorded in Unix epoch format.

3.4 Data Analysis Procedures

To formalize the evaluation metrics, let $N^{\text{repo}}_{\text{attempted}}$ and $N^{\text{repo}}_{\text{rejected}}$ denote the number of repository upload attempts and rejected uploads, and let $N^{\text{end}}_{\text{mal}}$, $N^{\text{end}}_{\text{blocked}}$, $N^{\text{end}}_{\text{legit}}$, and $N^{\text{end}}_{\text{false}}$ denote, respectively, the number of malicious packages presented to end users, malicious packages blocked, legitimate packages tested, and legitimate packages incorrectly rejected.

Additionally, let k_i denote the number of malicious artifacts successfully uploaded in trial i before key or certificate expiration, and let T denote the total number of repository-side trials. We define the following evaluation metrics:

$$\text{PreventionRate}_{\text{repo}} = \frac{N^{\text{repo}}_{\text{rejected}}}{N^{\text{repo}}_{\text{attempted}}} \times 100\%, \quad (1)$$

$$\text{DetectionRate}_{\text{end}} = \frac{N^{\text{end}}_{\text{blocked}}}{N^{\text{end}}_{\text{mal}}} \times 100\%, \quad (2)$$

$$\text{FalsePositiveRate}_{\text{end}} = \frac{N^{\text{end}}_{\text{false}}}{N^{\text{end}}_{\text{legit}}} \times 100\%, \quad (3)$$

$$K_{\text{total}} = \sum_{i=1}^{T} k_i. \quad (4)$$

We also measure detection latency and verification latency (in milliseconds), as well as attack-type detection breakdown across the four attack scenarios and detection mechanism analysis identifying which Sigstore components triggered failures. Results are tabulated by configuration type and theft delay categories. Data analysis employed Python 3.11 with pandas for data manipulation and statistical calculations. Results were cross-tabulated by configuration and package type producing 2×2 contingency tables showing verification outcomes. The deterministic nature of cryptographic operations ensured identical results upon replication.

3.5 Assumptions and Limitations

This subsection goes over several assumptions and limitations in this experiment.

3.5.1 Assumptions

This study assumes that Sigstore's cryptographic primitives provide computationally infeasible resistance to forgery, attackers have compromised individual publisher credentials but not the Certificate Authority (Fulcio) or transparency log (Rekor)—reflecting realistic scenarios where developers are compromised but core infrastructure remains secure, and system clocks are synchronized and timestamps cannot be manipulated.

3.5.2 Limitations

Key limitations include: controlled simulated environment rather than production repositories, limiting generalizability but enabling exact replication, modest sample size (12 repository trials, 90 end-user trials) adequate for demonstrating conceptual effectiveness but insufficient for robust statistical inference, attack scenario coverage limited to four major types documented in literature, latency measurements in local environments without network round-trip times to real Sigstore services—measured latencies represent lower bounds rather than production expectations, and simulated data rather than real-world package transactions.

Despite limitations, the controlled experimental design enables clear demonstration of security mechanisms' effectiveness under ideal conditions, with the understanding that production deployments will address additional operational considerations. The replicable experimental procedures allow future research to extend findings with field studies or larger-scale simulations.

3.6 Validity and Reliability

This subsection discusses the validity and reliability of the experiment.

Internal Validity: The experimental design ensures strong internal validity through controlled manipulation of the independent variable (security configuration: baseline versus defense) while holding all other factors constant. Identical attack scenarios were used across both configurations, with only the key type (traditional versus ephemeral) or verification completeness varying. Automated data collection eliminated human error and observer bias.

External Validity: While generalizability is limited by the simulated environment, attack scenarios were derived from actual supply chain incidents documented in threat intelligence reports. Experimental parameters reflect production recommendations and realistic threat distributions. The verification procedures replicate actual Sigstore operations, ensuring technical fidelity.

Construct Validity: Security constructs are operationalized through measurable outcomes aligned with security engineering principles. Detection effectiveness is operationalized as prevention and detection rates. Usability is operationalized as false positive rate on legitimate packages, addressing concerns from prior Sigstore adoption studies (Kalu et al., 2025).

Reliability All measurements were collected automatically through instrumented code with deterministic outcomes. Cryptographic verification produces identical results when repeated with the same inputs. Five trials from each experiment were re-run independently—all verification outcomes and latency measurements reproduced exactly. The complete experimental codebase is provided in supplementary materials enabling exact replication by other researchers.

3.7 Ethical Assurances

This study does not involve human subjects for the research. All experimental procedures were conducted in isolated simulation environments using synthetic data. No real repositories, actual developer credentials, or end users were involved. The attack simulations involved programmatically generated malicious packages existing only in the experimental environment. The research follows principles of research integrity by providing complete methodological transparency and enabling exact replication through published code. The experimental code includes attack simulation components that replicate techniques already documented in public security literature (Sonatype, 2024; Liu et al., 2020). The defensive measures validated provide significant security improvements outweighing any potential misuse risks from simulation methodology.

This section presented the methodology for empirically evaluating Sigstore-based security measures through two complementary experiments. The quantitative experimental design compared baseline configurations (traditional key

pairs) against defense configurations (Sigstore ephemeral keys, transparency logs, and identity-based authorization) across repository and end-user contexts. The repository-side experiment tested whether ephemeral keys prevent malicious uploads after key compromise using 12 trials with varying credential theft timing. The end-user experiment tested whether comprehensive Sigstore verification detects malicious packages using 90 trials covering four major attack types plus legitimate controls. Data collection employed fully automated instrumentation ensuring consistency and replicability. The methodology acknowledges simulated environment limitations while maintaining strong internal validity through controlled experimental design and high reliability through deterministic cryptographic operations. All procedures were designed for exact replication with complete code provided in supplementary materials.

4 Findings

This work provides a possible mitigation to attacks happening at the very beginning of the software supply chain –the open-source repository—and the very end of the software supply chain –the end user –by incorporating Sigstore into the security measures on both ends. In the process of reviewing previous literature, several key findings can be deduced.

The first key finding is that as highlighted in under the key management problem section, ephemeral key security measures can make it easier to harden the security of data transfer between open-source repositories. Because of the high management overhead involved with traditional key-pairs (Merril et al. 2023), many open-source repositories and the publishers uploading to them alike do not implement best practice security measures. This is evidently causing security vulnerabilities across sectors; the 2025 Synopsys report found that 86% of analyzed codebases had open-source vulnerabilities, with the highest industry of internet and mobile apps being 100% (Synopsys, 2025). Even energy and clean tech, the industry with the lowest rate of having open-source vulnerabilities is still 60% (Synopsys, 2025). While it cannot be said for certain that the challenges of key-pair management are the sole cause for these security vulnerabilities, previous literature such the one by Kalu, et al, highlighting that as the main issues in web PKI, and another study by Merril et al. emphasizing the problems that come with a compromised traditional key-pair, it proves to have a strong correlation (Kalu, et al 2025; Merril et al. 2023). The use of ephemeral keys can mitigate this key management problem entirely since developers would no longer need to keep their private key secure since the key only lasts a short period of time. Furthermore, Sigstore specifically has ephemeral key signing all integrated providing even less overhead. It should be emphasized once more that an ephemeral key-pair may still be compromised. However, the experiment conducted in this study demonstrated that when ephemeral keys with a 10-minute time-to-live are implemented alongside Key Authorization Maps, a 55.6% prevention rate of malicious uploads is achieved, compared to 0% prevention with traditional long-lived keys, validating that the time-limited nature of ephemeral credentials significantly constrains the window of vulnerability available to attackers, as shown in fig 1 and 2 below. In practice it is unlikely that a key would get stolen within 10 minutes, so this is evidently highly effective at preventing malicious uploads by stolen keys.

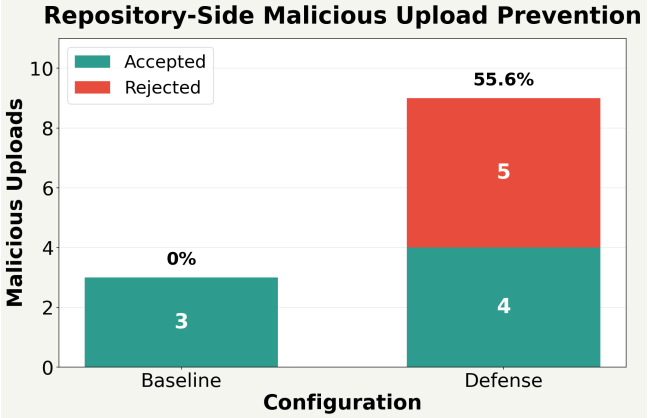


Fig. 4. Repository-side prevention comparison: Baseline vs. Sigstore defense with ephemeral keys and Key Authorization Maps

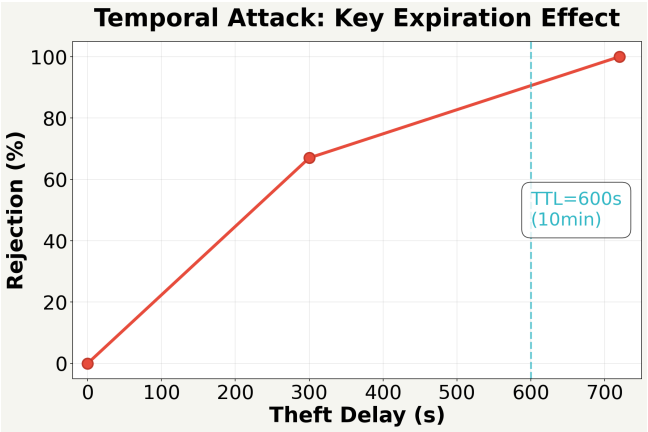


Fig. 5. Temporal attack window showing how ephemeral key expiration (TTL=600s) increases prevention rate over time

In addition to the ephemeral key hashing being integrated into Sigstore, the verification with a transparency log is also already integrated. This feature would address the issue of the repository provenance itself. As highlighted in the earlier problem: verification section, even if proper key-pair measures are put in place, in standard practice there lacks a sufficient verification of those keys and their signatures (Schorlemmer et al. 2024). This lack of verification could allow malicious artifacts to get through into the open-source repository and introduce security risks to the end user. With Sigstore's easily implementable verification feature, the end user themselves would also be able to verify whatever artifact they are about to use as well. So even

if a malicious artifact was somehow able to get through the repository's Sigstore-integrated security measures via compromised package, backdated package, malicious mirror, or typosquatting, end-users could also do a final verification on their end to make sure they do not face security breaches when utilizing open-sourced software. This would greatly increase the security provenance of the software supply chain. Currently, most developers who incorporate open-sourced software from repositories do not actually verify the signatures of artifacts (Kalu, et al, 2025). This could be attributed to the end-users (i.e developers) having too much trust in the repositories they are sourcing the artifacts from. As highlighted several times in this work, current open-source repository standards' security measures do not warrant this trust. Even if repositories were to implement signature verification, with the ease of Sigstore's implementation, end-users would benefit greatly from doing this verification themselves as well. Having verification would provide a greatly increased security measure —as demonstrated by this study, end-user verification achieved 100% detection of malicious packages across all attack scenarios tested (compromised packages, rollback attacks, malicious mirrors, and typosquatting), while maintaining zero false positives on legitimate packages, demonstrated in figure 3.

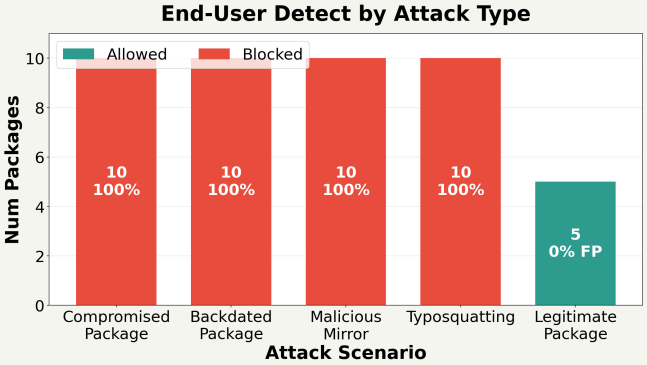


Fig. 6. End-user verification results showing 100% detection of all malicious attack types with zero false positives on legitimate packages

However, it is important to note that as emphasized in Section 3, the end-user verification cannot detect stolen keys; this specific attack vector must be prevented on the repository-side. This is because the end-user measures verify the certificates on the Rekor log, and anything signed with a stolen key would appear to have valid credentials. The repository side measures implement ephemeral keys with a short window of only ten minutes of valid keys. Although the repository-side experiment only had a 55.6% prevention rate, this result included simulated stolen keys within the ten-minute time window. In practice it could be deduced that an attacker stealing a key in ten minutes is unlikely, although possible if the publisher has a compromised OIDC login or an OIDC login that could be cracked within ten minutes. Therefore a highly secure OIDC login is essential in preventing ephemeral keys being compromised. Despite the aforementioned issues that are impeding the current widespread adoption of Sigstore across sectors in the respective Lack of Sigstore Adoption section, with this work addressing several of Sigstore's issues directly, and other works addressing others (Merril et al 2023), Sigstore is sufficient to be adopted as the standard for encrypted open-source data transfer. With the ease of implementation, key components such as ephemeral key signatures, issuance of certificates, and easy verification of signatures all integrated into a single toolset, this work aims to set a rigorous standard in open-source incorporated software supply chain security. As highlighted earlier in this work, current standards do not meet similar rigor to that of web data transfer security, especially standards the NIST sets (National Institute of Standards and Technology 2020). A study conducted over 140 million FireFox release users found that 92.1% of them use https for the initial browser request for a website (Kerschbaumer, et al. 2025). However, a study conducted on several open-source repositories found that in repositories such as npm and PyPI, less than 0.5% of distributed artifacts were signed (Schorlemmer et al. 2024). The lack of adequate security measures highlighted in open-source repositories highlighted have evidently introduced more security risks to end users wishing to use artifacts from them. If the same security standards are set for open-source software supply chains, security risks would be mitigated by a large degree.

5 Discussion and Conclusions

This work highlighted the problem of current insufficient security measures in the software supply chain, in particular the open-source repository and end-user sides. From the difficulty of key management (source) to the lack of verification of signatures present in current security standards (source), thorough literature review revealed just how inadequate current security measures are in practice. The stringent security practices of web PKI are evidently lacking in the security measures of the software supply chain, which introduces great concerns for security provenance, especially with how widespread open-sourced software use is.

Sigstore, an open-source set of tools for increasing security in software supply chains, was introduced in this work as a possible solution for the aforementioned security risks. This work first conducted a rigorous literature review on current software supply chain security practices, then on detailed literature regarding Sigstore itself. Subsequently a practical experiment was conducted comparing current security practices, to a more robust security measure incorporating Sigstore along with a KAM. This experiment was conducted on both a simulated repository and simulated end users. The results show that Sigstore is highly effective at reducing security breaches, with a 55.6% prevention rate of malicious artifacts on the repository side, and a 100% detection/block rate of malicious artifacts on the end-user side.

Despite Sigstore's effectiveness, a rigorous security standard using it is not currently present in sectors using open-sourced software. One major factor that is preventing the wide-spread adoption of Sigstore is the detection latency.

Layer	Config	Malicious Attacks	Prevented	Successful	Prevention Rate	Detection Latency
Repository	Baseline	3	0	3	0%	Immediate (upload)
Repository	Defense (Sigstore)	9	5	4	55.6%	Immediate (upload)
End-User	Baseline	40	0	40	0%	∼0.02 ms
End-User	Defense (Sigstore)	40	40	0	100%	∼0.2 ms

TABLE I

MALICIOUS ARTIFACT PREVENTION, DETECTION, AND LATENCY RESULTS FOR EACH LAYER AND CONFIGURATION IN THE EXPERIMENT.

Although the detection latency in this work's experiment was minimal at less than one millisecond, real world adoption of Sigstore would be utilizing public Rekor logs rather than private ones, which would greatly increase the latency. As discussed in earlier sections many professional developers are hesitant to use Sigstore as part of their software supply chain security measures due to this latency (Kalu et al 2025). Since public Rekor logs are connected to such a large amount of users and openly accessible at any time –which is the entire point of Sigstore's Rekor logs, to have transparent logging of certificates- that also introduces latency issues which could pose security risks, especially as attackers are constantly playing a cat-and-mouse game of security-breaching techniques with current technology. If this work's intent of setting a sufficient security standard across sectors for software supply chain security is to be widely adopted, this latency issue must be addressed. Future work would be to conduct research on how to reduce this latency for public Rekor logs.

References

[1] de Sousa, J. O., Carvalho De Farias, B., Filho, E. B. D. L., & Cordeiro, L. (in press). Trust, but Verify: Evaluating Developer Behavior in Mitigating Security Vulnerabilities in Open-Source Software Projects. In Proceedings of the 24th Brazilian Symposium on Information and Computational Systems Security. Retrieved October 5, 2025, from https://pure.manchester.ac.uk/ws/portalfiles/portal/339472411/2408.14273v1.pdf

[2] Kabir, M. M. A., Wang, Y., Yao, D. (D.), & Meng, N. (2022). How do developers follow security-relevant best practices when using NPM packages? In Proceedings of the IEEE Cybersecurity Development (SecDev), 77–83. https://doi.org/10.1109/SecDev53368.2022.00027

[3] Kalu, K. G., Okorafor, S., Singla, T., Chen, S., Torres-Arias, S., & Davis, J. C. (2025). Why Johnny signs with next-generation tools: A usability case study of Sigstore. arXiv. https://doi.org/10.48550/arXiv.2503.00271

[4] Kalu, K. G., Singla, T., Okafor, C., Torres-Arias, S., & Davis, J. C. (2025, August). An industry interview study of software signing for supply chain security. In Proceedings of the 34th USENIX Security Symposium. USENIX Association. https://www.usenix.org/system/files/conference/usenixsecurity25/sec25cycle1-prepub-232-kalu.pdf

[5] Kerschbaumer, C., Keeler, M., & Barnes, R. (2025, February). The state of HTTPS adoption on the web. In Proceedings of the 8th Workshop on Measurements, Attacks, and Defenses for the Web (MADWeb '25). Mozilla Research. https://research.mozilla.org/files/2025/03/the_state_of_https_adoption_on_the_web.pdf

[6] Kwon, B., Hong, S., Jeon, Y., & Kim, D. (2021). Certified malware in South Korea: A localized study of breaches of trust in code-signing PKI ecosystem. In Proceedings of the 23rd International Conference on Information and Communications Security (pp. 59-77). Springer. https://doi.org/10.1007/978-3-030-86890-1_4

[7] Liu, P., Ji, S., Fu, L., Lu, K., Zhang, X., Lee, W.-H., Lu, T., Chen, W., & Beyah, R. (2020). Understanding the security risks of Docker Hub. Proceedings of the 2020 ACM SIGSAC Conference on Computer and Communications Security, 1–17. https://doi.org/10.1145/3372297.3423342

[8] Merrill, K., Newman, Z., Torres-Arias, S., & Sollins, K. R. (2023, November 26–30). Speranza: Usable, privacy-friendly software signing. In Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (CCS '23) (pp. 3196–3209). Association for Computing Machinery. https://doi.org/10.1145/3576915.3623200

[9] National Institute of Standards and Technology. (2020). Recommendation for key management—Part 1: General (NIST Special Publication 800-57 Part 1 Revision 5). https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r5.pdf

[10] National Security Agency & Cybersecurity and Infrastructure Security Agency. (2023, June). Defending Continuous Integration/Continuous Delivery (CI/CD) Environments. U.S. Department of Defense. https://media.defense.gov/2023/Jun/28/2003249466/-1/-1/0/CSI_DEFENDING_CI_CD_ENVIRONMENTS.PDF

[11] Newman, Z., Meyers, J. S., & Torres-Arias, S. (2022, November 7). Sigstore: Software signing for everybody. In Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS '22) (pp. 2353–2367). ACM. https://doi.org/10.1145/3548606.3560596

[12] Okafor, C., Torres-Arias, S., & Zou, X. (2024). An Industry Interview Study of Software Signing for Supply Chain Security. arXiv preprint arXiv:2406.08198v3. https://arxiv.org/html/2406.08198v3

[13] Rekow, P. T., Egele, M., & Bhargavan, K. (2021). A holistic analysis of web-based public key infrastructure failures. Journal of Cybersecurity, 7(1), tyab025. https://doi.org/10.1093/cybsec/tyab025. https://academic.oup.com/cybersecurity/article/7/1/tyab025/6470936?login=false

[14] Ruohonen, J., Hjerppe, K., & Rindell, K. (2021). A large-scale security-oriented static analysis of Python packages in PyPI. arXiv preprint arXiv:2107.12699. https://arxiv.org/abs/2107.12699

[15] Schorlemmer, T. R., Kalu, K. G., Chigges, L., Ko, K. M., Isghair, E. A.-M., Baghi, S., Torres-Arias, S., & Davis, J. C. (2024). Signing in four public software package registries: Quantity, quality, and influencing factors (arXiv preprint arXiv:2401.14635). https://doi.org/10.48550/arXiv.2401.14635

[16] Sonatype. (2024, October 10). 10th Annual State of the Software Supply Chain Report. Sonatype. https://www.sonatype.com/state-of-the-software-supply-chain/introduction

[17] Synopsys. (2025). 2025 Open Source Security and Risk Analysis Report. Synopsys. https://www.blackduck.com/content/dam/black-duck/en-us/reports/rep-ossra.pdf